PyTorch for Training LLMs from Scratch

PyTorch for Training LLMs from Scratch----------1
Chapter 1: Introduction to PyTorch and LLMs---------- 3
1.1 What is PyTorch?---------- 3
1.2 Why PyTorch for LLM Training?----------4
1.3 What You'll Learn----------4
1.4 Prerequisites---------- 4
1.5 Setting Up Your Environment---------- 5
Chapter 2: Understanding Tensors---------- 6
2.1 What are Tensors?----------6
2.2 Tensors vs NumPy Arrays---------- 7
2.3 When to Use Tensors vs NumPy----------7
Chapter 3: Tensor Creation and Basic Operations---------- 8
3.1 Creating Tensors from Data---------- 8
3.1.1 From Python Lists---------- 8
3.1.2 Specifying Data Types----------10
3.2 Creating Tensors with Specific Values---------- 11
3.2.1 Zeros, Ones, and Empty Tensors---------- 11
3.2.2 Identity and Diagonal Matrices---------- 12
3.3 Random Tensor Creation----------13
3.4 Creating Ranges and Sequences----------13
Chapter 4: Tensor Indexing and Slicing----------14
4.1 Basic Indexing----------14
4.2 Advanced Slicing---------- 16
4.3 Boolean Indexing (Masking)----------17
4.4 Fancy Indexing----------18
Chapter 5: Mathematical Operations on Tensors---------- 18
5.2 Aggregation Operations---------- 21
5.3 Matrix Operations---------- 22
5.4 Broadcasting----------25
Chapter 6: GPU Acceleration with CUDA6.1 Understanding CPU vs GPU----------28
6.3 Moving Tensors to GPU---------- 30
6.4 Best Practices for GPU Usage----------31
6.5 GPU Performance Considerations----------32
Chapter 7: Tensor Shapes and Broadcasting7.1 Understanding Tensor Shapes----------33
7.2 Reshaping Tensors---------- 34
7.3 Expanding and Squeezing Dimensions----------35
7.4 Permuting and Transposing---------- 36

7.5 Concatenation and Stacking----------37
Chapter 8: Memory Management and Performance---------- 40
8.1 Understanding Contiguous Memory----------40
8.2 In-Place Operations----------41
8.3 Memory Optimization Techniques----------42
8.4 Profiling and Benchmarking---------- 44
Chapter 9: The Engine – torch.autograd and Backpropagation----------46
9.1 The Computational Graph----------46
9.2 Tracking Gradients: requires_grad----------46
9.3 The Backward Pass---------- 47
9.4 Disabling Gradient Tracking (Inference Mode)----------47
Chapter 10: The Skeleton – nn.Module----------49
10.1 Anatomy of a Module---------- 49
10.2 Building a Simple Feed-Forward Network (FFN)---------- 49
10.3 Understanding Parameters----------50
Chapter 11: The Goal – Loss Functions----------51
11.1 Common Loss Functions----------51
11.2 CrossEntropyLoss in LLMs----------51
Chapter 12: The Driver – Optimizers (torch.optim)---------- 53
12.1 Stochastic Gradient Descent (SGD) vs AdamW----------53
12.2 The Optimization Loop---------- 53
12.3 Learning Rate Schedulers----------54
Chapter 13: Putting it Together – A Complete Training Script---------- 55
Chapter 14: Vectorizing Language – nn.Embedding---------- 57
14.1 The Theory: The Lookup Table----------57
14.2 Implementing Embeddings---------- 57
14.3 Under the Hood: The Weights----------59
14.4 Positional Embeddings (Crucial for Transformers)----------60
Chapter 15: The Spark – Advanced Activation Functions---------- 62
15.1 Why Non-Linearity?---------- 62
15.2 The Modern Standard: GELU & SiLU---------- 62
15.3 Coding Activations----------62
15.4 Creating a Custom Activation (Swish from scratch)---------- 64
Chapter 16: Stabilizing Deep Networks – Normalization & Dropout---------- 65
16.1 Layer Normalization (nn.LayerNorm)----------65
16.2 RMSNorm (Root Mean Square Norm)---------- 66
16.3 Dropout (nn.Dropout)---------- 67
Chapter 17: Seeing Patterns – Convolutional Neural Networks (CNNs)----------69
17.1 The Convolution Operation----------69
17.2 nn.Conv2d (For Images)---------- 69
17.3 nn.Conv1d (For Text)---------- 70

17.4 Pooling Layers----------71
18.1 The Architecture Class---------- 73
18.2 Testing the Architecture---------- 74
Chapter 19: Advanced Weight Initialization----------75
19.1 PyTorch Defaults vs Custom----------75
19.2 torch.nn.init---------- 75
Chapter 20: Vectorizing Language – nn.Embedding---------- 76
20.1 The Theory: The Lookup Table----------76
20.2 Implementing Embeddings---------- 76
20.3 Under the Hood: The Weights----------77
20.4 Positional Embeddings (Crucial for Transformers)----------78
Chapter 21: The Spark – Advanced Activation Functions---------- 80
21.1 Why Non-Linearity?---------- 80
21.2 The Modern Standard: GELU & SiLU---------- 80
21.3 Coding Activations----------80
21.4 Creating a Custom Activation (Swish from scratch)---------- 81
Chapter 22: Stabilizing Deep Networks – Normalization & Dropout---------- 82
22.1 Layer Normalization (nn.LayerNorm)----------82
22.2 RMSNorm (Root Mean Square Norm)---------- 84
22.3 Dropout (nn.Dropout)---------- 84
Chapter 23: Seeing Patterns – Convolutional Neural Networks (CNNs)----------86
23.1 The Convolution Operation----------86
23.2 nn.Conv2d (For Images)---------- 86
23.3 nn.Conv1d (For Text)---------- 87
23.4 Pooling Layers----------88
Chapter 24: Practical Application – Building a Text Classifier---------- 89
24.1 The Architecture Class---------- 89
24.2 Testing the Architecture---------- 91
Chapter 25: Advanced Weight Initialization----------92
25.1 PyTorch Defaults vs Custom----------92
25.2 torch.nn.init---------- 92
Chapter 26 - Conclusion----------93

Chapter 1: Introduction to PyTorch and LLMs

1.1 What is PyTorch?

PyTorch is an open-source machine learning framework developed by Meta AI (formerly Facebook AI Research). It provides:

1. **Dynamic Computational Graphs**: Unlike static frameworks, PyTorch builds graphs on-the-fly during execution
2. **Python-First Design**: Seamless integration with Python ecosystem
3. **Automatic Differentiation**: Built-in autograd system for computing gradients
4. **GPU Acceleration**: Native CUDA support for high-performance computing
5. **Rich Ecosystem**: Extensive libraries for computer vision, NLP, and more

1.2 Why PyTorch for LLM Training?

Large Language Models (LLMs) like GPT, LLAMA, and Claude are trained using PyTorch because:

1. **Flexibility**: Dynamic graphs allow for variable-length sequences and complex architectures
2. **Debugging**: Pythonic nature makes debugging easier than compiled frameworks
3. **Research-Friendly**: Quick prototyping and experimentation
4. **Production-Ready**: Can be optimized and deployed at scale
5. **Community Support**: Vast resources, pre-trained models, and active development

1.3 What You'll Learn

By the end of this comprehensive guide, you will be able to:

1. Master all PyTorch components needed for deep learning
2. Understand transformer architecture from the ground up
3. Implement attention mechanisms and positional encodings
4. Build and train a complete LLM from scratch
5. Handle large-scale training with optimization techniques
6. Generate text using your trained model

1.4 Prerequisites

Required Knowledge:

1. Python programming (intermediate level)
2. Basic linear algebra (matrices, vectors, dot products)
3. Understanding of calculus (derivatives, chain rule)

4. Basic neural network concepts (optional but helpful)

Software Requirements:

```
None
# Install PyTorch (CUDA 11.8 version)
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118

# Install additional libraries
pip install numpy matplotlib jupyter tokenizers datasets
transformers
```

1.5 Setting Up Your Environment

```python
# Verify PyTorch installation

import torch
import sys

print(f"Python version: {sys.version}")
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"CUDA version: {torch.version.cuda}")
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"GPU Count: {torch.cuda.device_count()}")
```

Expected Output:

```
None
```

```
Python version: 3.10.x
PyTorch version: 2.1.0
CUDA available: True
CUDA version: 11.8
GPU: NVIDIA GeForce RTX 3090
GPU Count: 1
```

Chapter 2: Understanding Tensors

2.1 What are Tensors?

Tensors are the fundamental data structure in PyTorch. They are multi-dimensional arrays similar to NumPy's `ndarray` but with additional capabilities:

1. **GPU Acceleration**: Can be moved to GPU for faster computation
2. **Automatic Differentiation**: Track operations for gradient computation
3. **Optimized Operations**: Highly optimized C++/CUDA backend

Tensor Dimensions:

1. **Scalar** (0D): Single number → `tensor(5.0)`
2. **Vector** (1D): Array of numbers → `tensor([1, 2, 3])`
3. **Matrix** (2D): 2D array → `tensor([[1, 2], [3, 4]])`
4. **3D Tensor**: Common for images (channels, height, width)
5. **4D Tensor**: Batch of images (batch, channels, height, width)
6. **5D+ Tensors**: Video data, sequences of images, etc.

2.2 Tensors vs NumPy Arrays

Feature PyTorch Tensor NumPy Array

Feature	Option 1	Option 2
GPU Support	✅ Yes	❌ No
Automatic Differentiation	✅ Yes	❌ No
Deep Learning Operations	✅ Built-in	❌ Limited
Speed (CPU)	Similar	Similar
Speed (GPU)	⚡ Very Fast	N/A

2.3 When to Use Tensors vs NumPy

Use PyTorch Tensors when:

1. Training neural networks
2. Need gradient computation
3. Require GPU acceleration
4. Working with deep learning models

Use NumPy when:

1. Simple numerical computations
2. Data preprocessing (can convert to tensors later)
3. Working with libraries that require NumPy

Chapter 3: Tensor Creation and Basic Operations

3.1 Creating Tensors from Data

3.1.1 From Python Lists

```python
import torch


# Scalar tensor
scalar = torch.tensor(3.14)
print(f"Scalar: {scalar}")
print(f"Shape: {scalar.shape}")
print(f"Dimensions: {scalar.ndim}")


# Vector tensor
vector = torch.tensor([1, 2, 3, 4, 5])
print(f"Vector: {vector}")
print(f"Shape: {vector.shape}")
print(f"Dimensions: {vector.ndim}")


# Matrix tensor
matrix = torch.tensor([[1, 2, 3],
                       [4, 5, 6],
                       [7, 8, 9]])
print(f"Matrix:\n{matrix}")
print(f"Shape: {matrix.shape}")
print(f"Dimensions: {matrix.ndim}")


# 3D tensor
tensor_3d = torch.tensor([[[1, 2], [3, 4]],
                          [[5, 6], [7, 8]]])
print(f"3D Tensor:\n{tensor_3d}")
```

```python
print(f"Shape: {tensor_3d.shape}")
print(f"Dimensions: {tensor_3d.ndim}")
```

Output:

```
None
Scalar: tensor(3.1400)
Shape: torch.Size([])
Dimensions: 0

Vector: tensor([1, 2, 3, 4, 5])
Shape: torch.Size([5])
Dimensions: 1

Matrix:
tensor([[1, 2, 3],
      [4, 5, 6],
      [7, 8, 9]])
Shape: torch.Size([3, 3])
Dimensions: 2

3D Tensor:
tensor([[[1, 2],
       [3, 4]],
      [[5, 6],
       [7, 8]]])
Shape: torch.Size([2, 2, 2])
Dimensions: 3
```

3.1.2 Specifying Data Types

```python
# Default is float32 for decimal numbers
import torch

float_tensor = torch.tensor([1.0, 2.0, 3.0])
print(f"Default float: {float_tensor.dtype}") # torch.float32

# Explicitly specify dtype
int_tensor = torch.tensor([1, 2, 3], dtype=torch.int64)
print(f"Int64 tensor: {int_tensor.dtype}")

# Common data types
double_tensor = torch.tensor([1.0, 2.0], dtype=torch.float64) #
More precision
half_tensor = torch.tensor([1.0, 2.0], dtype=torch.float16) #
Less memory
bool_tensor = torch.tensor([True, False, True], dtype=torch.bool)

print(f"Float64 (double): {double_tensor.dtype}")
print(f"Float16 (half): {half_tensor.dtype}")
print(f"Boolean: {bool_tensor.dtype}")

# Type conversion
x = torch.tensor([1, 2, 3]) # Default int64
y = x.float() # Convert to float32
z = x.double() # Convert to float64

print(f"Original: {x.dtype}")
print(f"Float: {y.dtype}")
print(f"Double: {z.dtype}")
```

3.2 Creating Tensors with Specific Values

3.2.1 Zeros, Ones, and Empty Tensors

```python
# Zeros tensor
zeros = torch.zeros(3, 4) # 3x4 matrix of zeros
print(f"Zeros:\n{zeros}")

# Ones tensor
ones = torch.ones(2, 3, 4) # 2x3x4 tensor of ones
print(f"Ones shape: {ones.shape}")

# Empty tensor (uninitialized memory - values are random)
empty = torch.empty(2, 3)
print(f"Empty (random values):\n{empty}")

# Full tensor (filled with a specific value)
full = torch.full((3, 3), 7.5)
print(f"Full (all 7.5):\n{full}")
```

3.2.2 Identity and Diagonal Matrices

```python
# Identity matrix
identity = torch.eye(4)
print(f"Identity matrix:\n{identity}")

# Diagonal matrix
diagonal = torch.diag(torch.tensor([1, 2, 3, 4]))
print(f"Diagonal matrix:\n{diagonal}")

# Create tensor with same shape as another
x = torch.tensor([[1, 2], [3, 4]])
zeros_like = torch.zeros_like(x)
```

```
ones_like = torch.ones_like(x)
print(f"Zeros like x:\n{zeros_like}")
print(f"Ones like x:\n{ones_like}")
```

3.3 Random Tensor Creation

Random tensors are crucial for:

1. Initializing neural network weights
2. Data augmentation
3. Stochastic processes
4. Monte Carlo simulations

```python
# Random values from uniform distribution [0, 1)
import torch

rand_uniform = torch.rand(3, 3)
print(f"Uniform [0, 1):\n{rand_uniform}")

# Random values from standard normal distribution (mean=0, std=1)
rand_normal = torch.randn(3, 3)
print(f"Normal (0, 1):\n{rand_normal}")

# Random integers
rand_int = torch.randint(low=0, high=10, size=(3, 3))
print(f"Random integers [0, 10):\n{rand_int}")

# Set random seed for reproducibility
torch.manual_seed(42)
reproducible1 = torch.randn(2, 2)

torch.manual_seed(42)
reproducible2 = torch.randn(2, 2)
```

```
print(f"First random tensor:\n{reproducible1}")
print(f"Second random tensor (same seed):\n{reproducible2}")
print(f"Are they equal? {torch.equal(reproducible1,
reproducible2)}")
```

3.4 Creating Ranges and Sequences

```python
import torch

# Similar to Python's range()
arange = torch.arange(0, 10, 2)  # Start, end (exclusive), step
print(f"Arange: {arange}")

# Linearly spaced values
linspace = torch.linspace(0, 1, 5)  # Start, end (inclusive),
number of points
print(f"Linspace: {linspace}")

# Logarithmically spaced values
logspace = torch.logspace(0, 3, 4)  # 10^0 to 10^3 with 4 points
print(f"Logspace: {logspace}")

# Meshgrid for 2D coordinates
x = torch.linspace(-2, 2, 5)
y = torch.linspace(-2, 2, 5)
xx, yy = torch.meshgrid(x, y, indexing='ij')
print(f"X grid shape: {xx.shape}")
print(f"Y grid shape: {yy.shape}")
```

Chapter 4: Tensor Indexing and Slicing

4.1 Basic Indexing

```python
Indexing in PyTorch is similar to NumPy and Python lists.

# Create sample tensor
import torch
x = torch.arange(0, 20).reshape(4, 5)

print(f"Original tensor:\n{x}")

# Single element access
print(f"\nElement [0, 0]: {x[0, 0]}")
print(f"Element [2, 3]: {x[2, 3]}")

# Row access
print(f"\nFirst row: {x[0]}")
print(f"Last row: {x[-1]}")

# Column access (all rows, specific column)
print(f"\nFirst column: {x[:, 0]}")
print(f"Third column: {x[:, 2]}")

# Multiple elements
print(f"\nFirst two rows:\n{x[:2]}")
print(f"Last three columns:\n{x[:, -3:]}")
```

Output:

```
None
Original tensor:
tensor([[ 0, 1, 2, 3, 4],
[ 5, 6, 7, 8, 9],
[10, 11, 12, 13, 14],
[15, 16, 17, 18, 19]])

Element [0, 0]: 0
Element [2, 3]: 13

First row: tensor([0, 1, 2, 3, 4])
Last row: tensor([15, 16, 17, 18, 19])

First column: tensor([ 0, 5, 10, 15])
Third column: tensor([ 2, 7, 12, 17])
```

4.2 Advanced Slicing

```python
x = torch.arange(0, 24).reshape(4, 6)
print(f"Original:\n{x}")

# Step slicing
print(f"Every other row:\n{x[::2]}")
print(f"Every other column:\n{x[:, ::2]}")

# Reverse order
print(f"Reversed rows:\n{x[::-1]}")
print(f"Reversed columns:\n{x[:, ::-1]}")
# Submatrix
print(f"Center 2x2:\n{x[1:3, 2:4]}")
```

```python
# Skip patterns

print(f"Every other element in each row:\n{x[:, ::2]}")
```

4.3 Boolean Indexing (Masking)

Boolean indexing is extremely useful for filtering data based on conditions.

```python
import torch

x = torch.randn(5, 5)

print(f"Original: {x}")

# Boolean mask
mask = x > 0
print(f"Positive values mask: {mask}")

# Get positive values
positive = x[mask]
print(f"Positive values only: {positive}")

# Multiple conditions
mask_complex = (x > -0.5) & (x < 0.5)
values_in_range = x[mask_complex]
print(f"Values between -0.5 and 0.5: {values_in_range}")

# Replace values
x[x < 0] = 0 # Set all negative values to 0 (ReLU operation!)
print(f"After ReLU: {x}")

# Practical example: Remove outliers
data = torch.randn(100) * 10
```

```
mean = data.mean()
std = data.std()
outlier_mask = (data > mean + 2*std) | (data < mean - 2*std)
clean_data = data[~outlier_mask] # ~ is NOT operator
print(f"Original data points: {len(data)}")
print(f"After removing outliers: {len(clean_data)}")
```

4.4 Fancy Indexing

```python
import torch

x = torch.arange(0, 20).reshape(4, 5)

print(f"Original:\n{x}")

# Index with tensor
indices = torch.tensor([0, 2])
print(f"Rows 0 and 2:\n{x[indices]}")

# Multiple dimensions
row_indices = torch.tensor([0, 1, 2])
col_indices = torch.tensor([1, 3, 4])
print(f"Elements at (0,1), (1,3), (2,4): {x[row_indices,
col_indices]}")

# Gather operation (common in LLMs for embedding lookup)
embeddings = torch.randn(10, 5) # 10 vocabulary items, 5
dimensions
indices = torch.tensor([2, 5, 7, 1]) # Select items 2, 5, 7, 1
selected = embeddings[indices]
print(f"Selected embeddings shape: {selected.shape}") # [4, 5]
```

Chapter 5: Mathematical Operations on Tensors

Python

```python
### **5.1 Element-wise Operations**


x = torch.tensor([1.0, 2.0, 3.0, 4.0])

y = torch.tensor([2.0, 2.0, 2.0, 2.0])


# Basic arithmetic


print(f"Addition: {x + y}")

print(f"Subtraction: {x - y}")

print(f"Multiplication: {x * y}")

print(f"Division: {x / y}")

print(f"Power: {x ** 2}")


# Trigonometric


angles = torch.tensor([0, torch.pi/4, torch.pi/2])


print(f"\nSine: {torch.sin(angles)}")
```

```python
print(f"Cosine: {torch.cos(angles)}")
print(f"Tangent: {torch.tan(angles)}")

# Exponential and logarithm

print(f"\nExponential: {torch.exp(x)}")
print(f"Natural log: {torch.log(x)}")
print(f"Log10: {torch.log10(x)}")
print(f"Square root: {torch.sqrt(x)}")

# Rounding operations

decimals = torch.tensor([1.2, 2.7, -0.5, -1.8])

print(f"\nFloor: {torch.floor(decimals)}")
print(f"Ceil: {torch.ceil(decimals)}")
print(f"Round: {torch.round(decimals)}")

# Clamp (clip values)

values = torch.tensor([-5, -2, 0, 3, 8, 12])
```

```
clamped = torch.clamp(values, min=-1, max=5)


print(f"\nOriginal: {values}")
print(f"Clamped [-1, 5]: {clamped}")
```

5.2 Aggregation Operations

```python
x = torch.randn(3, 4)


print(f"Tensor:\n{x}")


# Sum


print(f"\nTotal sum: {x.sum()}")
print(f"Sum along rows (axis=0): {x.sum(dim=0)}")
print(f"Sum along columns (axis=1): {x.sum(dim=1)}")


# Mean


print(f"\nMean: {x.mean()}")
print(f"Mean per column: {x.mean(dim=0)}")
```

```python
# Min and Max

print(f"\nMin: {x.min()}")
print(f"Max: {x.max()}")
print(f"Min per row: {x.min(dim=1)}")

# Argmin and Argmax (indices)

print(f"\nArgmax (global): {x.argmax()}")
print(f"Argmax per row: {x.argmax(dim=1)}")

# Standard deviation and variance

print(f"\nStd: {x.std()}")
print(f"Variance: {x.var()}")

# Median, mode, quantiles

print(f"\nMedian: {torch.median(x)}")
```

5.3 Matrix Operations

These operations are fundamental for neural networks!

```python
# Matrix multiplication


A = torch.randn(3, 4)

B = torch.randn(4, 5)

C = torch.matmul(A, B) # or A @ B


print(f"A shape: {A.shape}")

print(f"B shape: {B.shape}")

print(f"C = A @ B shape: {C.shape}") # (3, 5)


# Batch matrix multiplication (critical for transformers)


batch_A = torch.randn(10, 3, 4) # 10 matrices of 3x4

batch_B = torch.randn(10, 4, 5) # 10 matrices of 4x5

batch_C = torch.bmm(batch_A, batch_B) # Batch matrix multiply


print(f"\nBatch result shape: {batch_C.shape}") # (10, 3, 5)
```

```python
# Dot product (1D tensors)

v1 = torch.tensor([1.0, 2.0, 3.0])
v2 = torch.tensor([4.0, 5.0, 6.0])
dot = torch.dot(v1, v2)

print(f"\nDot product: {dot}")

# Transpose

matrix = torch.randn(3, 5)
transposed = matrix.T # or matrix.transpose(0, 1)

print(f"\nOriginal shape: {matrix.shape}")
print(f"Transposed shape: {transposed.shape}")

# Determinant and inverse (for square matrices)

square = torch.randn(3, 3)
det = torch.det(square)
inv = torch.inverse(square)
```

```python
print(f"\nDeterminant: {det}")
print(f"Inverse exists: {inv.shape == square.shape}")

# Eigenvalues and eigenvectors

eigenvalues, eigenvectors = torch.linalg.eig(square)

print(f"Eigenvalues shape: {eigenvalues.shape}")

# Matrix norm

norm = torch.linalg.norm(matrix)

print(f"\nFrobenius norm: {norm}")
```

5.4 Broadcasting

Broadcasting allows operations between tensors of different shapes by automatically expanding dimensions.

```python
# Example 1: Vector + Scalar


x = torch.tensor([1, 2, 3, 4])

y = 10

result = x + y


print(f"Vector + Scalar: {result}")


# Example 2: Matrix + Vector (broadcast across rows)


matrix = torch.ones(3, 4)

vector = torch.tensor([1, 2, 3, 4])

result = matrix + vector # vector is broadcast to each row


print(f"\nMatrix + Vector:\n{result}")


# Example 3: Different shaped matrices
```

```
A = torch.ones(3, 1) # Column vector
B = torch.ones(1, 4) # Row vector
C = A + B # Both are broadcast to (3, 4)

print(f"\n(3,1) + (1,4) = {C.shape}:\n{C}")

# Broadcasting rules visualization

a = torch.randn(4, 1, 8)
b = torch.randn( 5, 8)
c = a + b # broadcasts to (4, 5, 8)

print(f"\n(4,1,8) + (5,8) = {c.shape}")

# Common pattern in neural networks: adding bias

batch_size = 32
features = 128
activations = torch.randn(batch_size, features)
bias = torch.randn(features)
output = activations + bias # bias broadcast across batch
```

```
print(f"\nBatch activations + bias: {output.shape}")
```

Chapter 6: GPU Acceleration with CUDA6.1 Understanding CPU vs GPU

CPU (Central Processing Unit):

1. Few cores (4-32 typically)
2. High clock speed (3-5 GHz)
3. Good for sequential tasks
4. Versatile, handles any computation

GPU (Graphics Processing Unit):

1. Thousands of cores (2,000-10,000+)
2. Lower clock speed (1-2 GHz per core)
3. Excellent for parallel tasks
4. Specialized for matrix operations

Why GPUs for Deep Learning? Neural networks involve massive matrix multiplications that can be parallelized. A single GPU can be 10-100x faster than CPU for deep learning!**6.2 Checking GPU Availability**

```python
import torch

# Check if CUDA is available

print(f"CUDA Available: {torch.cuda.is_available()}")

# Get CUDA version

print(f"CUDA Version: {torch.version.cuda}")

# Number of GPUs

print(f"GPU Count: {torch.cuda.device_count()}")

# Current GPU

if torch.cuda.is_available():
```

```
    print(f"Current Device: {torch.cuda.current_device()}")
    print(f"Device Name: {torch.cuda.get_device_name(0)}")
    # Memory info
    print(f"Total Memory:
{torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
    print(f"Allocated: {torch.cuda.memory_allocated(0) / 1e9:.4f}
GB")
    print(f"Cached: {torch.cuda.memory_reserved(0) / 1e9:.4f}
GB")
```

6.3 Moving Tensors to GPU

```python
# Create tensor on CPU (default)

cpu_tensor = torch.randn(3, 3)

print(f"CPU Tensor device: {cpu_tensor.device}")

# Move to GPU

if torch.cuda.is_available():
    # Method 1: .cuda()
    gpu_tensor = cpu_tensor.cuda()
    print(f"GPU Tensor device: {gpu_tensor.device}")
    # Method 2: .to() - more flexible
    gpu_tensor2 = cpu_tensor.to('cuda')
    # Method 3: .to() with device object
    device = torch.device('cuda' if torch.cuda.is_available()
else 'cpu')
    gpu_tensor3 = cpu_tensor.to(device)
    # Create directly on GPU
    gpu_tensor4 = torch.randn(3, 3, device='cuda')
    # Move back to CPU
```

```
    cpu_again = gpu_tensor.cpu()
    # or
    cpu_again2 = gpu_tensor.to('cpu')
    print(f"Back to CPU: {cpu_again.device}")
```

6.4 Best Practices for GPU Usage

```python
# Define device at the start

device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')

print(f"Using device: {device}")

# Move all tensors to the same device

x = torch.randn(1000, 1000).to(device)
y = torch.randn(1000, 1000).to(device)

# Operations on GPU

result = x @ y # Much faster on GPU

print(f"Result device: {result.device}")

# Practical pattern for neural networks

class SimpleModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = torch.nn.Linear(100, 50)

    def forward(self, x):
```

```
        return self.linear(x)

# Create model and move to device

model = SimpleModel().to(device)

# Create input data on same device

input_data = torch.randn(32, 100).to(device)

# Forward pass (all on GPU)

output = model(input_data)

print(f"Output device: {output.device}")
```

6.5 GPU Performance Considerations

```python
import time

# Compare CPU vs GPU performance

size = 5000

# CPU timing

cpu_x = torch.randn(size, size)
cpu_y = torch.randn(size, size)

start = time.time()

cpu_result = cpu_x @ cpu_y

cpu_time = time.time() - start
```

```
print(f"CPU Time: {cpu_time:.4f} seconds")

# GPU timing

if torch.cuda.is_available():
    gpu_x = torch.randn(size, size, device='cuda')
    gpu_y = torch.randn(size, size, device='cuda')
    # Warm up GPU
    _ = gpu_x @ gpu_y
    # Synchronize to ensure operation completes
    torch.cuda.synchronize()
    start = time.time()
    gpu_result = gpu_x @ gpu_y
    torch.cuda.synchronize() # Wait for GPU to finish
    gpu_time = time.time() - start
    print(f"GPU Time: {gpu_time:.4f} seconds")
    print(f"Speedup: {cpu_time / gpu_time:.2f}x")

# Memory management

if torch.cuda.is_available():
    # Clear cache
    torch.cuda.empty_cache()
    # Monitor memory
    print(f"\nMemory allocated: {torch.cuda.memory_allocated(0) /
1e9:.4f} GB")
    print(f"Max memory allocated:
{torch.cuda.max_memory_allocated(0) / 1e9:.4f} GB")
    # Reset peak memory stats
    torch.cuda.reset_peak_memory_stats(0)
```

Chapter 7: Tensor Shapes and Broadcasting7.1 Understanding Tensor Shapes

```python
# Shape is crucial in deep learning!

tensor_1d = torch.randn(5)
tensor_2d = torch.randn(3, 4)
tensor_3d = torch.randn(2, 3, 4)
tensor_4d = torch.randn(10, 3, 28, 28) # Batch of images

print(f"1D shape: {tensor_1d.shape}") # torch.Size([5])
print(f"2D shape: {tensor_2d.shape}") # torch.Size([3, 4])
print(f"3D shape: {tensor_3d.shape}") # torch.Size([2, 3, 4])
print(f"4D shape: {tensor_4d.shape}") # torch.Size([10, 3, 28,
28])

# Get shape as tuple

print(f"\nShape as tuple: {tensor_4d.shape}")
print(f"Dimension 0 (batch): {tensor_4d.shape[0]}")
print(f"Dimension 1 (channels): {tensor_4d.shape[1]}")

# Total number of elements

print(f"\nTotal elements: {tensor_4d.numel()}") # 10 * 3 * 28 *
28
```

7.2 Reshaping Tensors

```python
# Original tensor

x = torch.arange(24)

print(f"Original shape: {x.shape}") # [24]
```

```python
# Reshape to 2D

x_2d = x.reshape(4, 6)

print(f"2D shape: {x_2d.shape}") # [4, 6]

# Reshape to 3D

x_3d = x.reshape(2, 3, 4)

print(f"3D shape: {x_3d.shape}") # [2, 3, 4]

# Use -1 to infer dimension

x_auto = x.reshape(4, -1) # -1 means "figure it out"

print(f"Auto shape: {x_auto.shape}") # [4, 6]

# View vs Reshape

# view() requires contiguous memory
# reshape() may create a copy if needed

x_view = x.view(4, 6) # No data copy
x_reshape = x.reshape(4, 6) # May copy data

print(f"View shares memory: {x_view.data_ptr() == x.data_ptr()}")
```

7.3 Expanding and Squeezing Dimensions

Python

```python
# Squeeze - remove dimensions of size 1

x = torch.randn(1, 3, 1, 5, 1)
```

```python
print(f"Original shape: {x.shape}") # [1, 3, 1, 5, 1]

squeezed = x.squeeze()

print(f"Squeezed: {squeezed.shape}") # [3, 5]

squeezed_dim = x.squeeze(0) # Squeeze only dimension 0

print(f"Squeeze dim 0: {squeezed_dim.shape}") # [3, 1, 5, 1]

# Unsqueeze - add dimension of size 1

x = torch.randn(3, 4)

print(f"\nOriginal: {x.shape}") # [3, 4]

unsqueezed_0 = x.unsqueeze(0)

print(f"Unsqueeze dim 0: {unsqueezed_0.shape}") # [1, 3, 4]

unsqueezed_2 = x.unsqueeze(2)

print(f"Unsqueeze dim 2: {unsqueezed_2.shape}") # [3, 4, 1]

# Negative indexing

unsqueezed_neg = x.unsqueeze(-1)

print(f"Unsqueeze dim -1: {unsqueezed_neg.shape}") # [3, 4, 1]
```

7.4 Permuting and Transposing

```python
# Transpose - swap two dimensions
```

```
x = torch.randn(3, 4, 5)

transposed = x.transpose(0, 1) # Swap dims 0 and 1

print(f"Original: {x.shape}") # [3, 4, 5]

print(f"Transposed: {transposed.shape}") # [4, 3, 5]

# Permute - rearrange any dimensions

x = torch.randn(2, 3, 4, 5) # [batch, channels, height, width]

permuted = x.permute(0, 2, 3, 1) # [batch, height, width,
channels]

print(f"\nOriginal: {x.shape}") # [2, 3, 4, 5]

print(f"Permuted: {permuted.shape}") # [2, 4, 5, 3]

# Common pattern: batch-first to sequence-first

# For RNNs: (batch, seq_len, features) -> (seq_len, batch,
features)

batch_first = torch.randn(32, 100, 512) # 32 samples, 100 time
steps, 512 features

seq_first = batch_first.permute(1, 0, 2)

print(f"\nBatch-first: {batch_first.shape}")

print(f"Sequence-first: {seq_first.shape}")
```

7.5 Concatenation and Stacking

```python
# Concatenate along existing dimension

x = torch.randn(2, 3)
y = torch.randn(2, 3)
z = torch.randn(2, 3)

# Concatenate along dimension 0 (rows)

cat_0 = torch.cat([x, y, z], dim=0)

print(f"Cat dim 0: {cat_0.shape}") # [6, 3]

# Concatenate along dimension 1 (columns)

cat_1 = torch.cat([x, y, z], dim=1)

print(f"Cat dim 1: {cat_1.shape}") # [2, 9]

# Stack creates new dimension

stack_0 = torch.stack([x, y, z], dim=0)

print(f"Stack dim 0: {stack_0.shape}") # [3, 2, 3]

stack_1 = torch.stack([x, y, z], dim=1)

print(f"Stack dim 1: {stack_1.shape}") # [2, 3, 3]

# Split tensor

tensor = torch.randn(12, 3)

splits = torch.split(tensor, 4, dim=0) # Split into chunks of
size 4
```

```python
print(f"\nNumber of chunks: {len(splits)}")

print(f"Each chunk shape: {splits[0].shape}") # [4, 3]

# Chunk - split into N chunks

chunks = torch.chunk(tensor, 3, dim=0)

print(f"Chunks: {len(chunks)}")

print(f"Each shape: {chunks[0].shape}") # [4, 3]
```

Chapter 8: Memory Management and Performance

8.1 Understanding Contiguous Memory

```python
# Contiguous means elements are stored sequentially in memory

x = torch.randn(3, 4)

print(f"Is contiguous: {x.is_contiguous()}") # True

# Transpose breaks contiguity

x_t = x.T

print(f"Transposed is contiguous: {x_t.is_contiguous()}") # False

# Make contiguous

x_cont = x_t.contiguous()

print(f"After contiguous(): {x_cont.is_contiguous()}") # True

# Why does this matter?

# Some operations require contiguous tensors
# Non-contiguous tensors may be slower

# View requires contiguous memory

x = torch.randn(3, 4)

y = x.transpose(0, 1)
```

```python
try:
    z = y.view(-1) # This will fail!
except RuntimeError as e:
    print(f"\nError: view() needs contiguous tensor")

# Solution: make it contiguous first

z = y.contiguous().view(-1)

print(f"Success: {z.shape}")
```

8.2 In-Place Operations

```python
# In-place operations modify tensors directly, saving memory

x = torch.randn(3, 3)

print(f"Original:\n{x}")

# Regular operation (creates new tensor)

y = x + 5

print(f"\nNew tensor ID: {id(y)}")
print(f"Original tensor ID: {id(x)}")

# In-place operation (modifies existing tensor)

x.add_(5) # Note the underscore!

print(f"\nAfter in-place add, ID: {id(x)}")
print(f"Modified:\n{x}")
```

```
# Common in-place operations

x = torch.randn(3, 3)
x.mul_(2) # x = x * 2
x.sub_(1) # x = x - 1
x.div_(3) # x = x / 3
x.clamp_(-1, 1) # x = clamp(x, -1, 1)
x.zero_() # x = 0
x.fill_(5) # x = 5

# Warning: In-place operations can break autograd!

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x * 2
# x.add_(1) # This would cause issues with gradients!
# y.backward() # Error!
```

8.3 Memory Optimization Techniques

```python
# 1. Use torch.no_grad() for inference

x = torch.randn(1000, 1000, requires_grad=True)

# Without no_grad (tracks gradients, uses more memory)

y = x * 2 + 5

# With no_grad (doesn't track gradients, saves memory)

with torch.no_grad():
    z = x * 2 + 5

print(f"Without no_grad, grad_fn: {y.grad_fn}")
```

```python
print(f"With no_grad, grad_fn: {z.grad_fn}") # None

# 2. Delete large tensors when done

large_tensor = torch.randn(10000, 10000)

# ... use tensor ...

del large_tensor
torch.cuda.empty_cache() # If using GPU

# 3. Use appropriate dtypes

float32 = torch.randn(1000, 1000, dtype=torch.float32) # 4 bytes
per element
float16 = torch.randn(1000, 1000, dtype=torch.float16) # 2 bytes
per element

print(f"Float32 memory: {float32.element_size() * float32.numel()
/ 1e6:.2f} MB")

print(f"Float16 memory: {float16.element_size() * float16.numel()
/ 1e6:.2f} MB")

# 4. Use .detach() to break computation graph

x = torch.randn(100, requires_grad=True)
y = x * 2
z = y.detach() # z is independent of computation graph

print(f"y requires_grad: {y.requires_grad}")

print(f"z requires_grad: {z.requires_grad}")
```

8.4 Profiling and Benchmarking

```python
import time

def benchmark_operation(operation, *args, num_runs=100):
    """Benchmark an operation"""
    # Warm up
    for _ in range(10):
        _ = operation(*args)
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    start = time.time()
    for _ in range(num_runs):
        _ = operation(*args)
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    elapsed = (time.time() - start) / num_runs
    return elapsed * 1000 # Convert to milliseconds

# Example: Compare different operations

x = torch.randn(1000, 1000)
y = torch.randn(1000, 1000)

time_matmul = benchmark_operation(torch.matmul, x, y)
time_mm = benchmark_operation(torch.mm, x, y)
time_at = benchmark_operation(lambda a, b: a @ b, x, y)

print(f"torch.matmul: {time_matmul:.3f} ms")
print(f"torch.mm: {time_mm:.3f} ms")
print(f"@ operator: {time_at:.3f} ms")

# PyTorch Profiler

from torch.profiler import profile, record_function,
ProfilerActivity
```

```
def model_forward(x):
    return torch.matmul(x, x.T)

x = torch.randn(1000, 1000)

with profile(activities=[ProfilerActivity.CPU],
             record_shapes=True) as prof:
    with record_function("model_forward"):
        model_forward(x)

print(prof.key_averages().table(sort_by="cpu_time_total",
row_limit=10))
```

Chapter 9: The Engine – `torch.autograd` and Backpropagation

To train an LLM, we need to calculate how much each parameter (weight) contributed to the error. This is done via differentiation. PyTorch handles this automatically using **Autograd**.

9.1 The Computational Graph

PyTorch builds a **Dynamic Computational Graph** (DAG) as you execute operations.

1. **Nodes:** Tensors (data).
2. **Edges:** Functions (operations).
3. **Leaves:** Input tensors (weights).
4. **Roots:** Output tensors (loss).

9.2 Tracking Gradients: requires_grad

By default, tensors do not track history. To make a tensor "trainable" (like the weights of an LLM), we set `requires_grad=True`.

```python
import torch

# 1. Create a tensor that tracks gradients (Simulating a Weight)
w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# 2. Operations (Forward Pass)
# y is a result of an operation on w, so it has a 'grad_fn'
y = w + 2
z = y * y * 2
out = z.mean()

print(f"y grad_fn: {y.grad_fn}") # AddBackward
print(f"z grad_fn: {z.grad_fn}") # MulBackward
print(f"out value: {out.item()}")
```

9.3 The Backward Pass

The magic happens when you call `.backward()`. PyTorch traverses the graph in reverse (from output to input) to calculate derivatives using the Chain Rule.

```python
# 3. Backward Pass
out.backward()

# 4. Inspect Gradients (d(out)/dw)
# The gradients are accumulated in the .grad attribute of the
leaf tensor
print(f"Gradients of w: \n{w.grad}")

# Mathematical Verification:
# out = 1/3 * \sum ( (w_i + 2)^2 * 2 )
# d(out)/dw_i = 4/3 * (w_i + 2)
# For w=1: 4/3 * (3) = 4. Correct.
```

9.4 Disabling Gradient Tracking (Inference Mode)

When generating text with an LLM (Inference), we don't need gradients. Tracking them wastes massive amounts of memory.

Context Managers:

```python
x = torch.randn(3, requires_grad=True)

# METHOD 1: torch.no_grad() (Standard for Inference)
with torch.no_grad():
    y = x * 2
    print(f"y requires_grad: {y.requires_grad}") # False

# METHOD 2: torch.inference_mode() (Newer, slightly faster,
strict immutability)
with torch.inference_mode():
    z = x * 2
```

Curator's Note: In LLM training, `torch.no_grad()` is vital during the validation loop to prevent GPU OOM (Out of Memory) errors.

Chapter 10: The Skeleton – nn.Module

The `torch.nn` library is the Lego set of Deep Learning. The central class is `nn.Module`. Every layer (Linear, Attention, Normalization) and the entire Model itself must inherit from this.

10.1 Anatomy of a Module

A module consists of:

1. `__init__`: Define the layers (state).
2. `forward`: Define the flow of data (behavior).

10.2 Building a Simple Feed-Forward Network (FFN)

This is the standard component inside every Transformer block in an LLM.

```python
import torch.nn as nn




class SimpleFFN(nn.Module):

    def __init__(self, input_dim, hidden_dim):

        super().__init__()

        # 1. Define Layers

        # Linear layer: y = xA^T + b

        self.layer1 = nn.Linear(input_dim, hidden_dim)
```

```python
        self.relu = nn.ReLU()  # Non-linearity
        self.layer2 = nn.Linear(hidden_dim, input_dim)

    def forward(self, x):
        # 2. Define Data Flow
        # x shape: [Batch_Size, input_dim]
        out = self.layer1(x)
        out = self.relu(out)
        out = self.layer2(out)
        return out



# Instantiation
model = SimpleFFN(input_dim=768, hidden_dim=3072)  # Standard
   BERT-base sizes
print(model)
```

10.3 Understanding Parameters

You can inspect what exactly is being learned.

Python

```python
# View parameter names and shapes
```

```
for name, param in model.named_parameters():

    print(f"Layer: {name} | Size: {param.size()} | Trainable:
   {param.requires_grad}")




# Total parameters calculation (Essential for estimating LLM
   size)

total_params = sum(p.numel() for p in model.parameters())

print(f"Total Parameters: {total_params}")
```

Chapter 11: The Goal – Loss Functions

The Loss Function quantifies how "bad" the model's prediction is. The goal of training is to minimize this value.

11.1 Common Loss Functions

1. **MSELoss (Mean Squared Error):** Used for regression.
2. **CrossEntropyLoss:** Used for Classification and **LLM Training**.

11.2 CrossEntropyLoss in LLMs

In an LLM, we predict the next token from a vocabulary of size V. The output is a probability distribution.

Python
```
import torch

import torch.nn as nn
```

```python
# Scenario: Vocabulary size of 10. Batch size of 2.
vocab_size = 10
batch_size = 2

# Raw output from model (Logits) - Not probabilities yet!
logits = torch.randn(batch_size, vocab_size,
   requires_grad=True)

# The actual target tokens (Ground Truth)
targets = torch.tensor([3, 9])  # Token IDs

# Initialize Loss
criterion = nn.CrossEntropyLoss()

# Calculate Loss
loss = criterion(logits, targets)
print(f"Loss Value: {loss.item()}")

loss.backward()  # Compute gradients
```

Crucial Theory: nn.CrossEntropyLoss combines `nn.LogSoftmax()` and `nn.NLLLoss()` (Negative Log Likelihood) in one mathematically stable step. **Never** apply Softmax to your outputs before passing them to this function in PyTorch.

Chapter 12: The Driver – Optimizers (`torch.optim`)

The Optimizer takes the gradients calculated by Autograd and updates the weights to minimize the loss.

12.1 Stochastic Gradient Descent (SGD) vs AdamW

1. **SGD:** The classic. Simple, but slow convergence for complex landscapes.
2. **Adam:** Adaptive Moment Estimation. Good default.
3. **AdamW:** Adam with Weight Decay fix. **The industry standard for training LLMs (GPT, Llama, BERT).**

12.2 The Optimization Loop

This is the boilerplate code you will write thousands of times.

```python
import torch.optim as optim
import torch
import torch.nn as nn
# Note: SimpleFFN class must be defined or imported here for the code to run.

# 1. Define Model and Loss
model = SimpleFFN(10, 20)
criterion = nn.CrossEntropyLoss()

# 2. Define Optimizer
# lr = Learning Rate (Hyperparameter)
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)

# 3. Dummy Data
input_data = torch.randn(16, 10)  # Batch 16, Dim 10
target_labels = torch.randint(0, 10, (16,))
```

```
# --- THE TRAINING STEP ---
model.train()  # Set mode to train (enables Dropout, Batch Norm
updates)

# A. Forward Pass
predictions = model(input_data)
loss = criterion(predictions, target_labels)

# B. Zero Gradients
# We must clear previous gradients, otherwise they accumulate!
optimizer.zero_grad()

# C. Backward Pass (Calculate Gradients)
loss.backward()

# D. Update Weights
optimizer.step()

print(f"Updated weights successfully. Final Loss: {loss.item()}")
```

12.3 Learning Rate Schedulers

For LLMs, a constant learning rate is rarely used. We usually "warm up" the LR and then decay it.

```python
from torch.optim.lr_scheduler import CosineAnnealingLR
import torch.optim as optim
# Assuming 'optimizer' is defined from the previous section

scheduler = CosineAnnealingLR(optimizer, T_max=100)

# Inside the training loop, after optimizer.step():
scheduler.step()
print(f"Current LR: {scheduler.get_last_lr()}")
```

Chapter 13: Putting it Together – A Complete Training Script

Let's combine Tensor, Autograd, `nn.Module`, Loss, and Optimizer into a clean, functional script training a classifier.

```python
import torch
import torch.nn as nn
import torch.optim as optim


# --- Configuration ---
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
LR = 0.01
EPOCHS = 5


# --- 1. Data (Synthetic) ---
# X: [100 samples, 2 features]
X = torch.randn(100, 2).to(DEVICE)
# y: [100 labels (0 or 1)] based on which quadrant X is in
y = ((X[:, 0] > 0) & (X[:, 1] > 0)).long().to(DEVICE)


# --- 2. Model ---
class BinaryClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 8),
            nn.ReLU(),
            nn.Linear(8, 2)  # Output 2 logits (Class 0, Class 1)
        )

    def forward(self, x):
        return self.net(x)
```

```
model = BinaryClassifier().to(DEVICE)


# --- 3. Setup ---
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=LR)


# --- 4. Training Loop ---
print(f"Training on {DEVICE}...")


for epoch in range(EPOCHS):
    # Forward
    logits = model(X)
    loss = loss_fn(logits, y)

    # Backward
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 1 == 0:
        # Calculate Accuracy
        preds = torch.argmax(logits, dim=1)
        acc = (preds == y).float().mean()
        print(f"Epoch {epoch + 1} | Loss: {loss.item():.4f} |
Acc: {acc:.2f}")


print("Training Complete.")
```

Chapter 14: Vectorizing Language – nn.Embedding

Before an LLM can understand text, that text must be converted into numbers.

While one-hot encoding works for small data, it is computationally impossible for large vocabularies (e.g., GPT-4 has ~100k tokens).

14.1 The Theory: The Lookup Table

An embedding layer is essentially a learnable lookup table. It maps an integer index (representing a word/token) to a dense vector of floating-point numbers.

1. **Input:** Indices $[Batch, Seq_Len]$ (Integers)
2. **Output:** Vectors $[Batch, Seq_Len, Embedding_Dim]$ (Floats)

Unlike fixed representations (like Word2Vec), PyTorch embeddings are initialized randomly and **learned** during training via backpropagation.

14.2 Implementing Embeddings

The module is `torch.nn.Embedding`.

```python
import torch

import torch.nn as nn



# Configuration

vocab_size = 10000 # Size of our dictionary

embedding_dim = 512 # Size of the vector representing each
word
```

```
# 1. Create the Embedding Layer

# This creates a matrix of shape [10000, 512]

embedding_layer = nn.Embedding(num_embeddings=vocab_size,
   embedding_dim=embedding_dim)


# 2. Input Data (Batch of 2 sentences, 4 tokens each)

# Indices must be LongTensor (integers)

input_indices = torch.tensor([

    [1, 5, 99, 2], # "The cat sat down"

    [45, 2, 8, 0] # "Hello down there <PAD>"

])


# 3. Forward Pass

vectorized_output = embedding_layer(input_indices)


print(f"Input Shape: {input_indices.shape}") # [2, 4]

print(f"Output Shape: {vectorized_output.shape}") # [2, 4,
   512]


# Checking the statistics

print(f"Mean of embedding for token ID 1:
   {vectorized_output[0,0].mean().item()}")
```

14.3 Under the Hood: The Weights

The weights of an embedding layer are just a standard Tensor (`requires_grad=True`). You can access and modify them.

```python
# Accessing the weight matrix

weights = embedding_layer.weight

print(f"Weight Matrix Shape: {weights.shape}") # [10000, 512]



# Practical Tip: Handling Padding

# When batching sentences of different lengths, we use a PAD
   token (usually index 0).

# We tell the layer to ignore this token (gradient is always
   zero for it).

safe_embedding = nn.Embedding(vocab_size, embedding_dim,
   padding_idx=0)



test_input = torch.tensor([[10, 0, 0]]) # Word + 2 Pads

out = safe_embedding(test_input)



# The vector for index 0 will remain all zeros

print(f"Padding Vector Sum: {out[0, 1].sum()}") # Should be
   0.0
```

14.4 Positional Embeddings (Crucial for Transformers)

Transformers process all tokens simultaneously (in parallel). They have no inherent concept of "order." We must *inject* order by adding Positional Embeddings to the Token Embeddings

The final embedding is equal to the token embedding plus the position embedding.

```python
class EmbeddingsWithPosition(nn.Module):

    def __init__(self, vocab_size, max_seq_len, embed_dim):

        super().__init__()

        self.token_embed = nn.Embedding(vocab_size, embed_dim)

        # Learnable position embeddings (GPT style)

        self.pos_embed = nn.Embedding(max_seq_len, embed_dim)


    def forward(self, x):

        # x shape: [batch, seq_len]

        seq_len = x.size(1)

        # Create position indices: [0, 1, 2, ..., seq_len-1]

        pos_indices = torch.arange(seq_len,
device=x.device).unsqueeze(0)
```

```
        # Look up embeddings
        tok_vecs = self.token_embed(x)
        pos_vecs = self.pos_embed(pos_indices)
        # Broadcast add: [B, S, D] + [1, S, D]
        return tok_vecs + pos_vecs


# Example Usage
model = EmbeddingsWithPosition(vocab_size=1000,
   max_seq_len=512, embed_dim=64)
input_data = torch.randint(0, 1000, (4, 50)) # Batch 4, Seq 50
output = model(input_data)
print(f"Transformer Input Shape: {output.shape}") # [4, 50,
   64]
```

Chapter 15: The Spark – Advanced Activation Functions

In Part 2, we used ReLU. While ReLU is great, modern LLMs (like Llama 3, GPT-4, PaLM) use smoother variants like **GELU** or **Swish (SiLU)**.

15.1 Why Non-Linearity?

Without activation functions, a neural network, no matter how deep, is mathematically equivalent to a single linear layer. Activations allow the network to learn complex curves and decision boundaries.

15.2 The Modern Standard: GELU & SiLU

1. **ReLU (Rectified Linear Unit):** $\max(0, x)$. Fast, but suffers from "Dying ReLU" (gradients die for negative inputs).
2. **GELU (Gaussian Error Linear Unit):** Used in GPT-2, GPT-3, BERT. It's a smoother approximation of ReLU that allows small negative values, aiding gradient flow.
3. **SiLU (Sigmoid Linear Unit) / Swish:** Used in Llama and PaLM. $x \cdot \sigma(x)$.

15.3 Coding Activations

Python

```python
import torch.nn.functional as F

import matplotlib.pyplot as plt



x = torch.linspace(-5, 5, 100)



# 1. ReLU
```

```
y_relu = F.relu(x)


# 2. GELU (The LLM standard)
y_gelu = F.gelu(x)


# 3. SiLU (The Llama standard)
y_silu = F.silu(x)


# Visualization code (Run this locally to see the curve)
"""
plt.plot(x, y_relu, label='ReLU')
plt.plot(x, y_gelu, label='GELU')
plt.plot(x, y_silu, label='SiLU')
plt.legend()
plt.grid()
plt.title("LLM Activation Functions")
plt.show()
"""
```

15.4 Creating a Custom Activation (Swish from scratch)

Sometimes you need to implement a paper's math directly.

```python
class Swish(nn.Module):
    def forward(self, x):
        return x * torch.sigmoid(x)

# Verify against PyTorch implementation
custom_swish = Swish()
x = torch.randn(5)
print(f"Custom: {custom_swish(x)}")
print(f"PyTorch: {F.silu(x)}")
```

Chapter 16: Stabilizing Deep Networks – Normalization & Dropout

Training deep networks (LLMs have 30 to 100+ layers) is unstable. Gradients can vanish or explode. Normalization fixes this by forcing layer outputs to have a consistent distribution (mean=0, variance=1).

16.1 Layer Normalization (`nn.LayerNorm`)

Crucial for NLP. Unlike Batch Normalization (which normalizes across the batch dimension), Layer Normalization normalizes across the **feature** dimension for a single sample. This makes it independent of batch size, which is vital for text sequences.

Formula: $$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} \cdot \gamma + \beta$$ Where γ (weight) and β (bias) are learnable parameters.

Python

```python
# Setup

batch_size, seq_len, embed_dim = 2, 5, 4

x = torch.randn(batch_size, seq_len, embed_dim)



# Apply Layer Norm over the last dimension (the embedding
   dimension)

layer_norm = nn.LayerNorm(normalized_shape=embed_dim)



out = layer_norm(x)
```

```
# Verification

# The mean of the LAST dimension should be approx 0

print(f"Input Mean: {x[0,0,:].mean():.4f}")

print(f"Output Mean: {out[0,0,:].mean():.4f}") # Should be
   ~0.0000

print(f"Output Std: {out[0,0,:].std():.4f}") # Should be
   ~1.0000


# Inspecting Learnable Parameters

print(f"Gamma (Weight): {layer_norm.weight}") # Starts as 1s

print(f"Beta (Bias): {layer_norm.bias}") # Starts as 0s
```

16.2 RMSNorm (Root Mean Square Norm)

Pro Tip: Most modern LLMs (Llama 2/3, Gopher) use RMSNorm. It is LayerNorm *without* re-centering the mean (saves computation). PyTorch 2.1+ supports this natively, or we can build it.

Python

```
class RMSNorm(nn.Module):

    def __init__(self, dim, eps=1e-8):

        super().__init__()

        self.scale = dim ** -0.5

        self.eps = eps

        self.g = nn.Parameter(torch.ones(dim)) # Learnable
```

```
    Gain


     def forward(self, x):

         # Calculate norm without subtracting mean

         norm = torch.norm(x, dim=-1, keepdim=True) *
    self.scale

         return x / (norm + self.eps) * self.g


rms = RMSNorm(embed_dim)

out_rms = rms(x)
```

16.3 Dropout (`nn.Dropout`)

Dropout prevents overfitting by randomly "killing" neurons during training.

Key Behavior:

1. **Training:** Randomly zeros out elements with probability p. Scales remaining elements by $1/(1-p)$ to maintain magnitude.
2. **Eval (Inference):** Does nothing. Identity function.

```python
dropout = nn.Dropout(p=0.5)
x = torch.ones(5, 5)

# 1. Training Mode
dropout.train()
```

```python
out_train = dropout(x)
print("--- Training Mode (approx 50% zeros) ---")
print(out_train)

# 2. Evaluation Mode
dropout.eval()
out_eval = dropout(x)
print("\n--- Eval Mode (Original values preserved) ---")
print(out_eval)

# Warning: A common bug in LLM training is forgetting to toggle
model.eval() during validation/generation. This results in
inconsistent outputs because Dropout keeps applying noise.
```

Chapter 17: Seeing Patterns – Convolutional Neural Networks (CNNs)

While LLMs are primarily Transformers, CNNs are essential for:

1. **Tokenizer construction** (conceptually similar to sliding windows).
2. **Multimodal LLMs** (processing images before feeding to the LLM).
3. **Character-level models** or sequence feature extraction.

17.1 The Convolution Operation

A convolution slides a "kernel" (filter) over the input to detect local patterns.

1. **2D Conv:** Slides over (Height, Width). Used for Images.
2. **1D Conv:** Slides over (Time/Sequence). Used for **Text/Audio**.

17.2 `nn.Conv2d` (For Images)

Let's briefly cover 2D Convs as they are the foundation of computer vision.

Python
```python
# Input: [Batch, Channels, Height, Width]

# Example: 1 RGB image, 32x32 size

image = torch.randn(1, 3, 32, 32)



# Conv Layer

# in_channels=3 (RGB), out_channels=16 (16 filters),
   kernel_size=3 (3x3 square)

conv2d = nn.Conv2d(in_channels=3, out_channels=16,
   kernel_size=3, padding=1)
```

```python
output = conv2d(image)

print(f"Image Output: {output.shape}") # [1, 16, 32, 32]
```

17.3 `nn.Conv1d` (For Text)

In text, we convolve over the sequence length. This acts as an **N-gram detector**. A kernel size of 3 sees 3 words at a time.

```python
# Input Text format for Conv1d is often: [Batch, Channels (Embed Dim), Seq Len]

# Note: This is different from RNNs/Transformers which usually want [Batch, Seq Len, Embed Dim]

# We usually need to use .permute() to swap axes.


batch_size = 16

seq_len = 50

embed_dim = 100


# Text Input

x = torch.randn(batch_size, seq_len, embed_dim)


# Swap to [Batch, Embed, Seq] for Conv1d
```

```
x = x.permute(0, 2, 1)


# Conv Layer

# Detects patterns across 3 words (trigrams)

conv1d = nn.Conv1d(in_channels=embed_dim, out_channels=32,
   kernel_size=3, padding=1)


out = conv1d(x)


# Swap back for other layers if needed

out = out.permute(0, 2, 1)

print(f"Text Conv Output: {out.shape}") # [16, 50, 32]
```

17.4 Pooling Layers

Pooling reduces dimensionality by summarizing features (e.g., "Is this pattern present anywhere in the window?").

1. `nn.MaxPool1d` / `nn.MaxPool2d`: Takes the maximum value.
2. `nn.AvgPool1d` / `nn.AvgPool2d`: Takes the average.

```python
# Max Pool over a window of 2
pool = nn.MaxPool1d(kernel_size=2)

# Apply to previous Conv output (shape: [16, 32, 50])
pooled_out = pool(out.permute(0, 2, 1))
```

```
print(f"Pooled Output: {pooled_out.shape}") # [16, 32, 25]
(Sequence length halved)
```

Chapter 18: Practical Application – Building a Text Classifier

Let's combine Embedding, Conv1d, ReLU, Dropout, and Linear into a robust module. This mimics the architecture of early efficient text classifiers (like FastText or CNN-for-Sentence-Classification).

18.1 The Architecture Class

```python
class CNNTextClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes):
        super().__init__()
        # 1. Embedding: Maps ID -> Vector
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        # 2. Convolution: Extracts local features (tri-grams)
        self.conv = nn.Conv1d(in_channels=embed_dim,
out_channels=128, kernel_size=3)
        # 3. Activation
        self.relu = nn.ReLU()
        # 4. Global Pooling: "Did we see this feature anywhere in
the sentence?"
        # We will implement Global Max Pooling manually in
forward
        # 5. Regularization
        self.dropout = nn.Dropout(0.5)
        # 6. Output Head
        self.fc = nn.Linear(128, num_classes)

    def forward(self, x):
        # x: [Batch, Seq_Len]
        # Embed: [Batch, Seq_Len, Embed_Dim]
        x = self.embedding(x)
        # Permute for Conv: [Batch, Embed_Dim, Seq_Len]
        x = x.permute(0, 2, 1)
        # Conv + ReLU: [Batch, 128, New_Seq_Len]
        x = self.relu(self.conv(x))
        # Global Max Pooling
        # Take max over the sequence dimension (dim 2)
```

```
        x = torch.max(x, dim=2).values # Shape: [Batch, 128]
        # Dropout
        x = self.dropout(x)
        # Classification
        logits = self.fc(x) # [Batch, Num_Classes]
        return logits
```

18.2 Testing the Architecture

Always test your model with dummy data to verify shapes before training.

```python
# Configuration
V = 5000 # Vocab
D = 64 # Embed Dim
C = 2 # Classes (Positive/Negative)

model = CNNTextClassifier(V, D, C)

# Dummy Batch: 8 sentences, length 20
dummy_input = torch.randint(0, V, (8, 20))

output = model(dummy_input)

print(f"Model Output Shape: {output.shape}") # Expected: [8, 2]
print(f"Sample Logits: {output[0]}")
```

Chapter 19: Advanced Weight Initialization

How you start matters. If weights are initialized too large, gradients explode. Too small, they vanish.

19.1 PyTorch Defaults vs Custom

PyTorch layers have decent defaults, but for training deep LLMs, specific strategies (like Xavier or Kaiming initialization) are required.

19.2 `torch.nn.init`

```python
import torch.nn.init as init

layer = nn.Linear(100, 100)

# Method 1: Xavier (Glorot) Uniform
# Good for Sigmoid/Tanh activations
init.xavier_uniform_(layer.weight)

# Method 2: Kaiming (He) Normal
# Good for ReLU/GELU activations (The LLM standard)
init.kaiming_normal_(layer.weight, nonlinearity='relu')

# Method 3: Constant (Biases usually 0)
init.constant_(layer.bias, 0)

print("Weights initialized using Kaiming Normal.")

# Curator's Note: When training Transformers from scratch,
initialization is one of the most common sources of failure (loss
not converging). A common trick for GPT models is to initialize
the weights of the residual layers by $1/\sqrt{2 \times
\text{num\_layers}}$ to keep signal variance controlled deep in
the network.
```

Chapter 20: Vectorizing Language – `nn.Embedding`

Before an LLM can understand text, that text must be converted into numbers. While one-hot encoding works for small data, it is computationally impossible for large vocabularies (e.g., GPT-4 has ~100k tokens). The solution is **Embeddings**.

20.1 The Theory: The Lookup Table

An embedding layer is essentially a learnable lookup table. It maps an integer index (representing a word/token) to a dense vector of floating-point numbers.

1. **Input:** Indices `[Batch, Seq_Len]` (Integers)
2. **Output:** Vectors `[Batch, Seq_Len, Embedding_Dim]` (Floats)

Unlike fixed representations (like Word2Vec), PyTorch embeddings are initialized randomly and **learned** during training via backpropagation.

20.2 Implementing Embeddings

```python
# The module is torch.nn.Embedding.

import torch
import torch.nn as nn


# Configuration

vocab_size = 10000 # Size of our dictionary
embedding_dim = 512 # Size of the vector representing each word


# 1. Create the Embedding Layer

# This creates a matrix of shape [10000, 512]

embedding_layer = nn.Embedding(num_embeddings=vocab_size,
embedding_dim=embedding_dim)
```

```python
# 2. Input Data (Batch of 2 sentences, 4 tokens each)

# Indices must be LongTensor (integers)

input_indices = torch.tensor([
[1, 5, 99, 2], # "The cat sat down"
[45, 2, 8, 0] # "Hello down there <PAD>"
])


# 3. Forward Pass

vectorized_output = embedding_layer(input_indices)


print(f"Input Shape: {input_indices.shape}") # [2, 4]

print(f"Output Shape: {vectorized_output.shape}") # [2, 4, 512]


# Checking the statistics

print(f"Mean of embedding for token ID 1:
{vectorized_output[0,0].mean().item()}")
```

20.3 Under the Hood: The Weights

The weights of an embedding layer are just a standard Tensor (`requires_grad=True`). You can access and modify them.

```python
# Accessing the weight matrix
weights = embedding_layer.weight
```

```
print(f"Weight Matrix Shape: {weights.shape}") # [10000, 512]

# Practical Tip: Handling Padding
# When batching sentences of different lengths, we use a PAD
token (usually index 0).
# We tell the layer to ignore this token (gradient is always zero
for it).
safe_embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=0)

test_input = torch.tensor([[10, 0, 0]]) # Word + 2 Pads
out = safe_embedding(test_input)

# The vector for index 0 will remain all zeros
print(f"Padding Vector Sum: {out[0, 1].sum()}") # Should be 0.0
```

20.4 Positional Embeddings (Crucial for Transformers)

Transformers process all tokens simultaneously (in parallel). They have no inherent concept of "order." We must *inject* order by adding Positional Embeddings to the Token Embeddings.

$$E_{final} = E_{token} + E_{position}$$

```python
class EmbeddingsWithPosition(nn.Module):

    def __init__(self, vocab_size, max_seq_len, embed_dim):

        super().__init__()

        self.token_embed = nn.Embedding(vocab_size, embed_dim)

        # Learnable position embeddings (GPT style)
```

```
        self.pos_embed = nn.Embedding(max_seq_len, embed_dim)

    def forward(self, x):

        # x shape: [batch, seq_len]

        seq_len = x.size(1)

        # Create position indices: [0, 1, 2, ..., seq_len-1]

        pos_indices = torch.arange(seq_len,
device=x.device).unsqueeze(0)

        # Look up embeddings

        tok_vecs = self.token_embed(x)

        pos_vecs = self.pos_embed(pos_indices)

        # Broadcast add: [B, S, D] + [1, S, D]

        return tok_vecs + pos_vecs

# Example Usage

model = EmbeddingsWithPosition(vocab_size=1000, max_seq_len=512,
embed_dim=64)

input_data = torch.randint(0, 1000, (4, 50)) # Batch 4, Seq 50

output = model(input_data)

print(f"Transformer Input Shape: {output.shape}") # [4, 50, 64]
```

Chapter 21: The Spark – Advanced Activation Functions

In Part 2, we used `ReLU`. While ReLU is great, modern LLMs (like Llama 3, GPT-4, PaLM) use smoother variants like **GELU** or **Swish (SiLU)**.

21.1 Why Non-Linearity?

Without activation functions, a neural network, no matter how deep, is mathematically equivalent to a single linear layer. Activations allow the network to learn complex curves and decision boundaries.

21.2 The Modern Standard: GELU & SiLU

1. **ReLU (Rectified Linear Unit):** `max(0, x)`. Fast, but suffers from "Dying ReLU" (gradients die for negative inputs).
2. **GELU (Gaussian Error Linear Unit):** Used in GPT-2, GPT-3, BERT. It's a smoother approximation of ReLU that allows small negative values, aiding gradient flow.
3. **SiLU (Sigmoid Linear Unit) / Swish:** Used in Llama and PaLM. $x \cdot \sigma(x)$.

21.3 Coding Activations

```python
import torch.nn.functional as F
import matplotlib.pyplot as plt


x = torch.linspace(-5, 5, 100)


# 1. ReLU
y_relu = F.relu(x)


# 2. GELU (The LLM standard)
y_gelu = F.gelu(x)


# 3. SiLU (The Llama standard)
y_silu = F.silu(x)
```

```
# Visualization code (Run this locally to see the curve)

"""

plt.plot(x, y_relu, label='ReLU')

plt.plot(x, y_gelu, label='GELU')

plt.plot(x, y_silu, label='SiLU')

plt.legend()

plt.grid()

plt.title("LLM Activation Functions")

plt.show()

"""
```

21.4 Creating a Custom Activation (Swish from scratch)

Sometimes you need to implement a paper's math directly.

```python
class Swish(nn.Module):

  def forward(self, x):
    return x * torch.sigmoid(x)

# Verify against PyTorch implementation
```

```
custom_swish = Swish()
x = torch.randn(5)
print(f"Custom: {custom_swish(x)}")
print(f"PyTorch: {F.silu(x)}")
```

Chapter 22: Stabilizing Deep Networks – Normalization & Dropout

Training deep networks (LLMs have 30 to 100+ layers) is unstable. Gradients can vanish or explode. Normalization fixes this by forcing layer outputs to have a consistent distribution (mean=0, variance=1).

22.1 Layer Normalization (`nn.LayerNorm`)

Crucial for NLP. Unlike Batch Normalization (which normalizes across the batch dimension), Layer Normalization normalizes across the **feature** dimension for a single sample. This makes it independent of batch size, which is vital for text sequences.

The output value *y* is calculated by first normalizing the input value *x*. This normalization is done by subtracting the expected mean of *x* and then dividing by the square root of the variance of *x* plus a small stabilization constant, epsilon. This normalized result is then scaled by a learnable weight, gamma, and shifted by a learnable bias, beta. Gamma and beta are parameters that the model learns during training.

```python
# Setup
import torch
import torch.nn as nn

batch_size, seq_len, embed_dim = 2, 5, 4

x = torch.randn(batch_size, seq_len, embed_dim)
```

```
# Apply Layer Norm over the last dimension (the embedding
dimension)

layer_norm = nn.LayerNorm(normalized_shape=embed_dim)


out = layer_norm(x)


# Verification

# The mean of the LAST dimension should be approx 0

print(f"Input Mean: {x[0,0,:].mean():.4f}")

print(f"Output Mean: {out[0,0,:].mean():.4f}") # Should be
~0.0000

print(f"Output Std: {out[0,0,:].std():.4f}") # Should be ~1.0000


# Inspecting Learnable Parameters

print(f"Gamma (Weight): {layer_norm.weight}") # Starts as 1s

print(f"Beta (Bias): {layer_norm.bias}") # Starts as 0s
```

22.2 RMSNorm (Root Mean Square Norm)

Pro Tip: Most modern LLMs (Llama 2/3, Gopher) use RMSNorm. It is LayerNorm *without* re-centering the mean (saves computation). PyTorch 2.1+ supports this natively, or we can build it.

```python
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-8):
        super().__init__()
        self.scale = dim ** -0.5
        self.eps = eps
        self.g = nn.Parameter(torch.ones(dim)) # Learnable Gain

    def forward(self, x):
        # Calculate norm without subtracting mean
        norm = torch.norm(x, dim=-1, keepdim=True) * self.scale
        return x / (norm + self.eps) * self.g

rms = RMSNorm(embed_dim)
out_rms = rms(x)
```

22.3 Dropout (`nn.Dropout`)

Dropout prevents overfitting by randomly "killing" neurons during training.

Key Behavior:

1. **Training:** Randomly zeros out elements with probability p. Scales remaining elements by $1/(1-p)$ to maintain magnitude.
2. **Eval (Inference):** Does nothing. Identity function.

```python
dropout = nn.Dropout(p=0.5)
```

```python
x = torch.ones(5, 5)


# 1. Training Mode

dropout.train()

out_train = dropout(x)

print("--- Training Mode (approx 50% zeros) ---")

print(out_train)


# 2. Evaluation Mode

dropout.eval()

out_eval = dropout(x)

print("\
--- Eval Mode (Original values preserved) ---")

print(out_eval)
```

Warning: A common bug in LLM training is forgetting to toggle `model.eval()` during validation/generation. This results in inconsistent outputs because Dropout keeps applying noise.

Chapter 23: Seeing Patterns – Convolutional Neural Networks (CNNs)

While LLMs are primarily Transformers, CNNs are essential for:

1. **Tokenizer construction** (conceptually similar to sliding windows).
2. **Multimodal LLMs** (processing images before feeding to the LLM).
3. **Character-level models** or sequence feature extraction.

23.1 The Convolution Operation

A convolution slides a "kernel" (filter) over the input to detect local patterns.

1. **2D Conv:** Slides over (Height, Width). Used for Images.
2. **1D Conv:** Slides over (Time/Sequence). Used for **Text/Audio**.

23.2 `nn.Conv2d` (For Images)

```python
# Input: [Batch, Channels, Height, Width]
# Example: 1 RGB image, 32x32 size
image = torch.randn(1, 3, 32, 32)

# Conv Layer
# in_channels=3 (RGB), out_channels=16 (16 filters),
kernel_size=3 (3x3 square)
conv2d = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3,
padding=1)

output = conv2d(image)

print(f"Image Output: {output.shape}") # [1, 16, 32, 32]
```

23.3 `nn.Conv1d` (For Text)

In text, we convolve over the sequence length. This acts as an **N-gram detector**. A kernel size of 3 sees 3 words at a time.

Input Text format for Conv1d is often: [Batch, Channels (Embed Dim), Seq Len]
Note: This is different from RNNs/Transformers which usually want [Batch, Seq Len, Embed Dim]
We usually need to use .permute() to swap axes.

```python
batch_size = 16

seq_len = 50

embed_dim = 100

# Text Input

x = torch.randn(batch_size, seq_len, embed_dim)

# Swap to [Batch, Embed, Seq] for Conv1d

x = x.permute(0, 2, 1)

# Conv Layer

# Detects patterns across 3 words (trigrams)

conv1d = nn.Conv1d(in_channels=embed_dim, out_channels=32, kernel_size=3, padding=1)

out = conv1d(x)

# Swap back for other layers if needed

out = out.permute(0, 2, 1)
```

```
print(f"Text Conv Output: {out.shape}") # [16, 50, 32]
```

23.4 Pooling Layers

Pooling reduces dimensionality by summarizing features (e.g., "Is this pattern present anywhere in the window?").

1. `nn.MaxPool1d` / `nn.MaxPool2d`: Takes the maximum value.
2. `nn.AvgPool1d` / `nn.AvgPool2d`: Takes the average.

```
# Max Pool over a window of 2
pool = nn.MaxPool1d(kernel_size=2)

# Apply to previous Conv output (shape: [16, 32, 50])
pooled_out = pool(out.permute(0, 2, 1))
print(f"Pooled Output: {pooled_out.shape}") # [16, 32, 25] (Sequence length halved)
```

Chapter 24: Practical Application – Building a Text Classifier

Let's combine `Embedding`, `Conv1d`, `ReLU`, `Dropout`, and `Linear` into a robust module. This mimics the architecture of early efficient text classifiers (like FastText or CNN-for-Sentence-Classification).

24.1 The Architecture Class

```python
class CNNTextClassifier(nn.Module):

def __init__(self, vocab_size, embed_dim, num_classes):

super().__init__()

# 1. Embedding: Maps ID -> Vector

self.embedding = nn.Embedding(vocab_size, embed_dim)

# 2. Convolution: Extracts local features (tri-grams)

self.conv = nn.Conv1d(in_channels=embed_dim, out_channels=128,
kernel_size=3)

# 3. Activation

self.relu = nn.ReLU()

# 4. Global Pooling: "Did we see this feature anywhere in the
sentence?"

# We will implement Global Max Pooling manually in forward

# 5. Regularization

self.dropout = nn.Dropout(0.5)
```

```
# 6. Output Head
self.fc = nn.Linear(128, num_classes)
def forward(self, x):
# x: [Batch, Seq_Len]
# Embed: [Batch, Seq_Len, Embed_Dim]
x = self.embedding(x)
# Permute for Conv: [Batch, Embed_Dim, Seq_Len]
x = x.permute(0, 2, 1)
# Conv + ReLU: [Batch, 128, New_Seq_Len]
x = self.relu(self.conv(x))
# Global Max Pooling
# Take max over the sequence dimension (dim 2)
x = torch.max(x, dim=2).values # Shape: [Batch, 128]
# Dropout
x = self.dropout(x)
# Classification
logits = self.fc(x) # [Batch, Num_Classes]
return logits
```

24.2 Testing the Architecture

Always test your model with dummy data to verify shapes before training.

```python
# Configuration

V = 5000 # Vocab

D = 64 # Embed Dim

C = 2 # Classes (Positive/Negative)


model = CNNTextClassifier(V, D, C)


# Dummy Batch: 8 sentences, length 20

dummy_input = torch.randint(0, V, (8, 20))


output = model(dummy_input)


print(f"Model Output Shape: {output.shape}") # Expected: [8, 2]

print(f"Sample Logits: {output[0]}")
```

Chapter 25: Advanced Weight Initialization

How you start matters. If weights are initialized too large, gradients explode. Too small, they vanish.

25.1 PyTorch Defaults vs Custom

PyTorch layers have decent defaults, but for training deep LLMs, specific strategies (like Xavier or Kaiming initialization) are required.

25.2 `torch.nn.init`

```python
import torch.nn.init as init
import torch.nn as nn

# Assuming 'nn' is imported for the Linear layer, e.g., 'import torch.nn as nn'
layer = nn.Linear(100, 100)

# Method 1: Xavier (Glorot) Uniform
# Good for Sigmoid/Tanh activations
init.xavier_uniform_(layer.weight)

# Method 2: Kaiming (He) Normal
# Good for ReLU/GELU activations (The LLM standard)
init.kaiming_normal_(layer.weight, nonlinearity='relu')

# Method 3: Constant (Biases usually 0)
init.constant_(layer.bias, 0)
print("Weights initialized using Kaiming Normal.")
```

Curator's Note: When training Transformers from scratch, initialization is one of the most common sources of failure (loss not converging). A common trick for GPT models is to initialize the weights of the residual layers by $1/\sqrt{2 \times \text{num_layers}}$ to keep signal variance controlled deep in the network.

Chapter 26 - Conclusion

This comprehensive curriculum has equipped you with the **foundational principles and advanced, hands-on techniques** necessary to build, train, and scale a modern Large Language Model (LLM) from the ground up using **PyTorch**.

We started by laying a solid **mathematical and computational groundwork**, mastering **Tensors**—the fundamental data structure—and leveraging **GPU acceleration** for efficient processing. This led directly into the heart of neural network training: **Autograd** and the dynamic computational graph, which automatically handles gradient calculation via **backpropagation**.

The curriculum then shifted to assembling the required architecture:

- **The Blueprint:** We utilized the `nn.Module` framework to construct complex, layered models, selecting optimal **Loss Functions** (like CrossEntropyLoss) and high-performance **Optimizers** (like AdamW) to drive learning.
- **The Components:** We explored advanced layers crucial for handling language: **Embeddings** for numerical representation, stabilizing networks with **Layer Normalization**, ensuring generalization with **Dropout**, and incorporating modern non-linearities like **GELU/SiLU**.
- **The Data Engine:** We mastered the critical data pipeline, implementing **custom Datasets and DataLoaders** to handle text efficiency. Most importantly, we conquered the complexity of **dynamic padding, subword tokenization**, and the essential creation of **Causal and Padding Attention Masks**.

Finally, we reached the **core of the LLM**: the **Transformer Decoder Block**. We implemented the intricate **Multi-Head Self-Attention** mechanism and modern **SwiGLU Feed-Forward Networks**, culminating in the assembly of a complete, GPT-style architecture.

This journey concluded with the crucial step of scalability, introducing **Distributed Training** concepts, specifically **FSDP and bfloat16 Mixed Precision**, which are mandatory techniques for training models with billions of parameters.

By combining deep theoretical understanding with extensive, runnable code examples, you are now prepared to navigate the complexities of LLM development, moving beyond simple library use to truly **engineer state-of-the-art language models**.

Chapter 27: Essential LLM Research & Resources

To master LLM development, focus on these defining papers and practical coding resources.

---

1. Foundational Research Papers

These papers introduced the core architectures and scaling laws.

Concept	Paper Title	Key Takeaway
Transformer	**Attention Is All You Need** (2017)	**The foundation** of all modern LLMs (GPT, BERT). Introduced the Self-Attention mechanism.
Pre-training	**BERT** (2018)	Introduced the **Masked Language Model** objective for pre-training large models.
Generative Models	**GPT-1** (2018)	Established the **decoder-only** Transformer for causal language generation.
Scaling Laws	**Language Models are Few-Shot Learners (GPT-3)** (2020)	Showed that scaling models enables **in-context learning** (few-shot ability).
Compute Optimal	**Chinchilla Scaling Laws** (2022)	Defined the optimal balance between **model size and dataset size** for efficient training.

2. Alignment & Post-Training

How models are made useful and safe.

- **RLHF: Training language models to follow instructions with human feedback** (2022). The process used to align models like ChatGPT.
- **DPO: Direct Preference Optimization** (2023). A simpler, better alternative to RLHF.

3. Essential PyTorch Implementation Videos

- **Andrej Karpathy's "Neural Networks: Zero to Hero" Playlist** (https://www.youtube.com/playlist?list=PLAqhIrjkxbuWI23v9tabJ0R8B9D6J-7I4)
    - **Micrograd Video:** Builds the PyTorch Autograd engine from scratch to demystify backpropagation.
    - **Build GPT-2 Video:** Codes and trains the complete **decoder-only Transformer** from scratch in PyTorch—the ultimate practical reference.

4. Code and Performance References

- **nanoGPT GitHub:** (https://github.com/karpathy/nanogpt) Minimal, clean PyTorch code for the GPT architecture.
- **PyTorch Official Tutorials:** (https://pytorch.org/tutorials/) Use for advanced topics like **Distributed Training (FSDP)** and performance.
- **Hugging Face Documentation:** (https://huggingface.co/docs/transformers/index) Review for industry-standard, optimized implementations.